

Resumen teórico — Sistemas Operativos

Notas del curso 5 y 6: *Procesos, threads, concurrencia e IPC*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico de los capítulos 5 (*Procesos y threads*) y 6 (*Concurrencia e IPC*) de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se cubren las nociones de proceso y de thread, los estados por los que puede pasar una tarea, las llamadas al sistema asociadas a su gestión, los modos de ejecución del procesador, el mecanismo de interrupciones y traps, los cambios de contexto, la concurrencia y sus problemas clásicos (condiciones de carrera, secciones críticas), las primitivas de sincronización (locks, spinlocks, sleep locks, semáforos, monitores, variables de condición), los problemas asociados al uso de locks (*deadlock*, *livelock*), los modelos de comunicación por mensajes y los mecanismos IPC ofrecidos por UNIX. El objetivo es servir como material de estudio para preparar el Práctico 3 y el primer parcial: cada concepto se introduce con intuición, se formaliza cuando hace falta y se conecta con los demás para que el lector pueda razonar sobre el modelo completo del SO.

Índice

1. Introducción	2
2. Procesos y threads	3
2.1. Definiciones básicas	3
2.2. Tareas: la abstracción común	3
2.3. Atributos de un proceso	4
2.4. Layout virtual de memoria	4
2.5. Registros del thread y el calling convention	4
2.6. Llamadas al sistema para gestión de procesos	5
2.7. Estados de un proceso	5
2.8. Manejo de tareas en el kernel	6
3. Modos de ejecución, traps e interrupciones	7
3.1. Modo usuario y modo supervisor	7
3.2. Interrupciones, excepciones y traps	7
3.3. Pasos del hardware ante una interrupción	7
3.4. Implementación de syscalls	7
4. Cambios de contexto	8
4.1. El problema	8
4.2. Quantum y preemption	8
4.3. Operaciones <code>yield()</code> y <code>scheduler()</code>	8
4.4. El timer y el preemptive multitasking	9
4.5. Pasos de un context switch	9
4.6. Esquema mínimo de un kernel	9
5. Procesos zombie y el primer proceso del sistema	10

5.1. El par <code>exit/wait</code>	10
5.2. El primer proceso del sistema	10
6. Concurrency	11
6.1. Definiciones y por qué importa	11
6.2. Paralelismo vs. tiempo compartido	11
6.3. Recursos compartidos, regiones críticas y <code>race conditions</code>	11
7. Locks y exclusión mutua	12
7.1. Definición	12
7.2. Spinlocks: el primer intento	12
7.3. Solución 1: deshabilitar interrupciones	13
7.4. Solución 2: instrucciones atómicas	13
7.5. Spinlocks en multiprocesadores	13
7.6. Sleep locks	14
7.7. Modularidad y problemas del uso de locks	14
8. Solución de Peterson	15
9. Semáforos	15
9.1. Definición	15
9.2. El productor-consumidor con semáforos	15
10. Variables de condición y monitores	16
10.1. Variables de condición	16
10.2. Monitores	17
11. Mensajes y comunicación	18
11.1. Mensajes en lugar de memoria compartida	18
11.2. Micro-kernels	18
12. Futuros, promesas y concurrencia sin locks	18
12.1. Futuros y promesas	18
12.2. Estructuras lock-free y wait-free	19
13. IPC en UNIX	19
14. Resumen final	19

1. Introducción

Cuando escribimos un programa pensamos en una secuencia única de instrucciones que se ejecuta de principio a fin. La realidad de un sistema operativo moderno es muy distinta: en una computadora hay *decenas* de programas corriendo a la vez — el navegador, el editor de texto, el reloj de la barra de tareas, los servicios del sistema, el kernel mismo — y la CPU es un recurso limitado. Una de las funciones más importantes del SO es darle a cada programa la *ilusión* de tener una computadora dedicada, mientras se ocupa internamente de repartir la CPU entre todos.

Para hacer esto posible el SO necesita una serie de abstracciones:

- un **proceso** representa cada programa en ejecución;
- un **thread** es la unidad más chica que el scheduler del SO puede planificar;

- un **estado** describe en qué momento de su ciclo de vida está cada tarea;
- un **cambio de contexto** es la operación con la que el SO le saca la CPU a una tarea y se la da a otra;
- los mecanismos de **sincronización** y los de **IPC** permiten que las tareas cooperen sin pisarse.

Idea intuitiva

La intuición es: el SO simula varias computadoras virtuales sobre una única física. Para que funcione, tiene que poder *guardar el trabajo en curso* de una tarea, cambiar a otra, y luego retomar la primera exactamente donde la dejó.

2. Procesos y threads

2.1. Definiciones básicas

Definición

Un **proceso** es una instancia de un programa en ejecución. Un **programa** es un archivo ejecutable que reside en algún dispositivo de almacenamiento. Un proceso tiene al menos un thread.

Definición

Un **thread** (o hilo) es la mínima secuencia de instrucciones que el SO puede planificar para ejecutarse en una CPU.

Un programa *en disco* es estático: no consume CPU ni memoria del sistema. Solo cuando es *cargado* en memoria y se le asigna un thread de ejecución se convierte en un *proceso*. Esa distinción es fundamental: del mismo programa pueden existir muchos procesos ejecutando simultáneamente, cada uno con su propio estado.

Un proceso ejecuta el código en *modo usuario* (el modo de menor privilegio de la CPU) y solo puede acceder a su *propio espacio de memoria*. Esta protección la implementa el SO con la ayuda de la MMU del hardware: si una instrucción intenta tocar una dirección fuera del espacio del proceso, la CPU dispara una excepción.

2.2. Tareas: la abstracción común

Es conveniente abstraer las diferencias técnicas entre procesos y threads y llamar a ambos **tareas** (*tasks*). Tanto un proceso como un thread son tareas que el scheduler conoce, planifica y representa con un descriptor.

Un *proceso multithreaded* crea varios threads dentro de él mismo. Cada thread tiene su propia pila, pero todos comparten el espacio de memoria del proceso (el código, los datos globales, el heap y los descriptores de archivos abiertos).

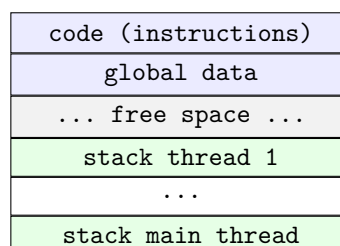


Figura 1: Imagen en memoria de un proceso multithreaded.

2.3. Atributos de un proceso

El SO mantiene, por cada proceso, un descriptor (a veces llamado *Process Control Block*, PCB) con al menos:

- un identificador único (`pid`);
- su estado actual;
- si está dormido, la razón o recurso por el cual lo está;
- su *mapa de memoria* accesible en modo usuario:
 - segmento de código (*text*);
 - datos globales inicializados (*data*) y sin inicializar (*bss*);
 - heap (memoria dinámica, manejada con `sbrk` o `malloc`);
 - stacks de cada thread.
- al menos un thread de ejecución, representado por su *stack en modo usuario* y su *stack en modo kernel*;
- el conjunto de recursos adquiridos (archivos abiertos, pipes, sockets, semáforos, áreas de memoria compartida, manejadores de señales, timers);
- el comando o programa del cual es instancia.

El *stack en modo kernel* merece atención especial: en él se guarda el *trapframe* (los valores de los registros del proceso al momento de una interrupción), se usa para las invocaciones a funciones del kernel mientras se ejecuta en el contexto del proceso y, finalmente, se guarda el *contexto* (registros caller-saved) del thread cuando ocurre un cambio de contexto.

2.4. Layout virtual de memoria

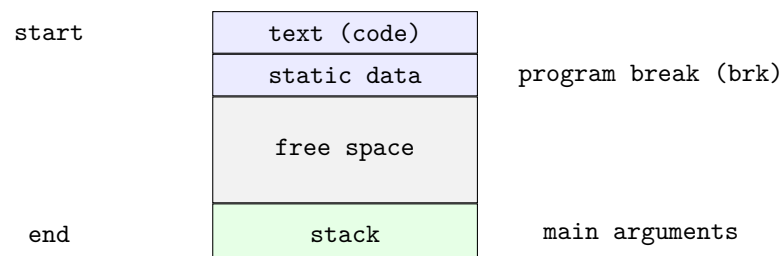


Figura 2: Layout virtual de la memoria de un proceso.

El *program break* (`brk`) marca el final del segmento de datos. La syscall `sbrk(n)` permite extender o reducir el área de datos asignando o liberando memoria al proceso (lo que usa `malloc()` internamente). Las direcciones `start` y `end` delimitan el espacio virtual del proceso; en muchos sistemas `start = 0`.

2.5. Registros del thread y el calling convention

El estado de un thread en un punto cualquiera de su ejecución está representado por los valores de los registros de la CPU. Tres son particularmente importantes:

- **Program counter** (`pc`): dirección de la próxima instrucción a ejecutar.
- **Stack pointer** (`sp`): tope de la pila.
- **Frame (base) pointer** (`fp`): dirección base del registro de activación de la función actual.

La pila contiene *registros de activación* (*activation records* o *stack frames*) para controlar la secuencia de invocación a funciones y sus retornos. Cada registro de activación contiene la dirección de retorno, el frame pointer salvado, los argumentos y las variables locales de la función. Esta organización es la que permite que las llamadas recursivas funcionen correctamente y que las variables locales se creen y destruyan automáticamente.

El **calling convention** de la ABI de la plataforma especifica cómo se pasan los argumentos (en registros, en pila, o en una combinación) y cómo se devuelve el valor de retorno. En RISC-V, por ejemplo:

- **x1/ra**: dirección de retorno;
- **x2/sp**: stack pointer;
- **x8/fp**: frame pointer;
- **x10–x17/a0–a7**: argumentos (los excedentes se apilan);
- **a0/a1**: valor de retorno;
- **s0–s11**: *saved registers* que la función invocada debe preservar.

2.6. Llamadas al sistema para gestión de procesos

Las syscalls clásicas de POSIX para administrar procesos son:

Syscall	Qué hace
<code>fork()</code>	Crea un proceso hijo, copia exacta del padre. El hijo hereda los descriptores de archivos abiertos.
<code>exec(path, args)</code>	Reemplaza la imagen del proceso (código y datos) por la del binario <code>path</code> . Si tiene éxito, no retorna.
<code>exit(exitstatus)</code>	Termina el proceso actual con el código <code>exitstatus</code> .
<code>wait(&status)</code>	El padre se bloquea hasta que termine alguno de sus hijos. Retorna el estado de salida en <code>status</code> .
<code>getpid()</code>	Retorna el identificador del proceso.
<code>kill(pid)</code>	Envía una señal al proceso con identificador <code>pid</code> .
<code>sleep(n)</code>	Bloquea al proceso por <code>n</code> segundos.
<code>sbrk(n)</code>	Asigna <code>n</code> bytes adicionales al proceso.

Idea intuitiva

`fork()` y `exec()` suelen ir juntos: el shell hace `fork()` para crear un hijo, y el hijo hace `exec()` del programa que el usuario quiere ejecutar. Mientras tanto, el padre se queda en `wait()` esperando a que el hijo termine. Esta es la base del modelo de procesos de UNIX.

2.7. Estados de un proceso

A lo largo de su vida, una tarea pasa por varios estados. Las Notas identifican los siguientes:

- **NEW**: recién creada, todavía no ingresó al scheduler.
- **RUNNING**: actualmente ejecutando, tiene asignada una CPU.
- **RUNNABLE** (o **READY**): lista para ejecutar, esperando que el kernel le asigne una CPU.
- **SLEEPING** (o **WAITING**): bloqueada hasta la ocurrencia de un evento que la despertará (por ejemplo, la finalización de una operación de I/O).
- **ZOMBIE**: terminó (`exit()`) pero el padre todavía no ejecutó `wait()` para recoger su estado de salida.
- **TERMINATED**: una vez que el padre hace `wait()`, el descriptor se libera.

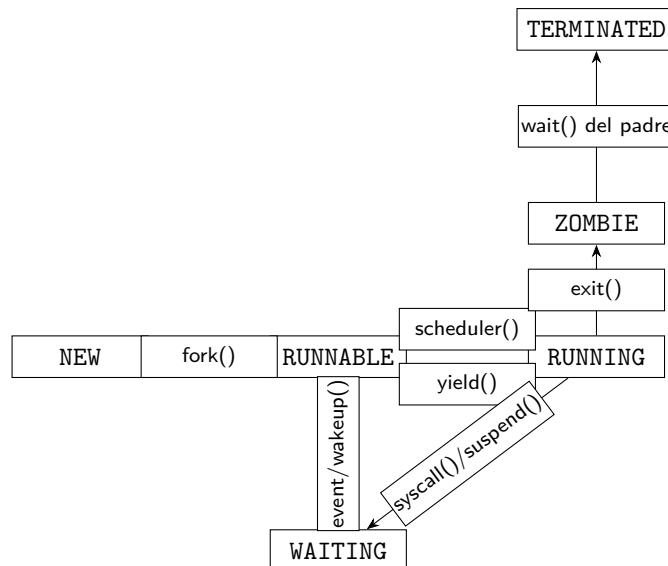


Figura 3: Cambios de estado de un proceso.

ZOMBIE y huérfanos. Un proceso que terminó pero todavía existe en la tabla del kernel se llama *zombie*. Esto sucede porque el SO debe preservar el código de salida hasta que el padre lo recoja con `wait()`. Si el padre termina antes que el hijo, el hijo queda *huérfano* y es *adoptado* por `init` (PID 1), que se encarga de recogerlo cuando termine.

2.8. Manejo de tareas en el kernel

El kernel organiza las tareas en estructuras de datos. Las tareas en estado **RUNNABLE** forman una cola (o varias colas, según el algoritmo de scheduling). Las tareas en **SLEEPING** se encolan en *wait queues* asociadas al evento o recurso por el que esperan: cuando llega el evento, el kernel desencola y despierta a la tarea correcta.

Idea intuitiva

Pensemos en una sala de espera del médico. La sala de listos (**RUNNABLE**) tiene a todos los pacientes que pueden ser atendidos. Las salas privadas (**wait queues**) tienen pacientes esperando un evento puntual: un análisis, una llamada, un resultado. Cuando el evento ocurre, el kernel los manda de vuelta a la sala de listos.

¿Qué pasa si el scheduler no encuentra ningún proceso **RUNNABLE**? Hay dos alternativas:

1. Quedarse en un ciclo de busca activa.
2. Tener un proceso *idle* creado al inicio que ejecuta `while (true) yield();`. Al obtener la CPU la libera inmediatamente, asegurando que siempre hay un **RUNNABLE** que elegir.

En cualquiera de los dos casos, este es el momento ideal para que el SO contabilice la *idleness* y dispare ahorro de energía o trabajo diferido (procesar paquetes de red, por ejemplo) en *kernel threads*.

3. Modos de ejecución, traps e interrupciones

3.1. Modo usuario y modo supervisor

Definición

Un proceso de usuario se ejecuta con la **CPU en modo usuario**. En este modo la CPU no puede ejecutar ciertas instrucciones privilegiadas (como deshabilitar interrupciones) y solo puede acceder a su propio espacio de memoria. El kernel se ejecuta en **modo kernel** (o *supervisor*), un modo de mayor privilegio sin restricciones.

Esta separación es la base de la protección: si un proceso pudiera deshabilitar interrupciones o tocar la memoria del kernel, podría hacer caer al sistema o monopolizar la CPU.

3.2. Interrupciones, excepciones y traps

Tres mecanismos llevan la CPU desde modo usuario a modo kernel:

- **Interrupciones:** las generan los dispositivos (teclado, discos, red) para notificar eventos (“terminé de leer”, “llegó un paquete”).
- **Excepciones:** las genera la CPU al intentar ejecutar una instrucción inválida (división por cero, acceso a memoria ilegal).
- **Traps:** las dispara intencionalmente un programa de usuario para invocar al kernel; es la base de las *llamadas al sistema*.

El SO instala, durante su inicialización, un **vector de interrupciones**: una tabla en memoria del kernel donde cada entrada apunta a una rutina (*interrupt service routine* o ISR) encargada de manejar el evento correspondiente.

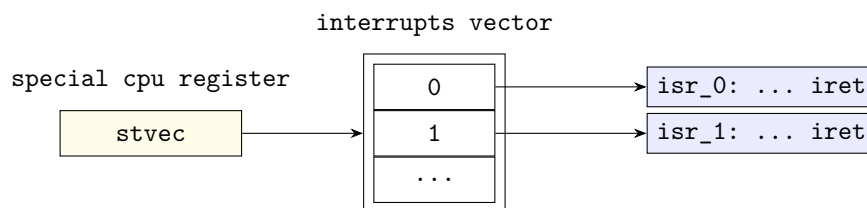


Figura 4: Vector de interrupciones.

3.3. Pasos del hardware ante una interrupción

1. Salva el **pc** actual en algún registro o en memoria.
2. Cambia el modo de ejecución a supervisor.
3. Salta a la rutina correspondiente según el número de interrupción (índice del vector).
4. Al retornar (instrucción **iret**), restaura el modo previo y el **pc** salvado.

En arquitecturas como RISC-V, todas las interrupciones disparan una única ISR cuya dirección está en **stvec**; el código del manejador identifica el motivo leyendo **scause** y despacha al caso correspondiente.

Importante

El kernel toma el control en cada interrupción, excepción o trap. Esa es la única manera de que las funciones del kernel se invoquen mientras un proceso de usuario está corriendo.

3.4. Implementación de syscalls

Las syscalls se implementan en dos partes:

- **Lado usuario:** típicamente como funciones en la biblioteca estándar. Cada función pone el *número de syscall* en un registro designado de la CPU (**a7** en RISC-V) y ejecuta una instrucción de trap (**int** en x86 o **ecall** en RISC-V).
- **Lado kernel:** el manejador de trap (ISR) reconoce que se trata de una syscall, lee el número y los argumentos del trapframe, y llama a la función correspondiente del kernel. El valor de retorno se escribe en el trapframe para que, al volver a usuario, esté en el registro adecuado.

Por convención POSIX, los valores negativos retornados por una syscall (típicamente **-1**) son códigos de error.

4. Cambios de contexto

4.1. El problema

Si una sola CPU debe ejecutar muchas tareas, el SO necesita poder *intercambiar* cuál de ellas tiene la CPU en cada momento. Para hacerlo correctamente, debe ser capaz de pausar a una tarea de modo que, más adelante, pueda retomarla *exactamente* donde la dejó.

Definición

El **saved context** de una tarea es la estructura que guarda el estado de la CPU desde que la tarea abandonó el procesador, para poder continuar su ejecución más adelante. Típicamente contiene el **pc**, el **sp** y posiblemente otros registros (los *callee-saved*).

El descriptor de cada tarea contiene su **tid**, su estado, sus recursos y su *saved context*.

4.2. Quantum y preemption

Definición

El **quantum** (o *time slice*) es el período de tiempo máximo de uso continuo de la CPU asignado a una tarea. Cuando una tarea agota su quantum, el kernel le *quita* la CPU (*preemption*) y se la asigna a otra **RUNNABLE**.

Idea intuitiva

El quantum hace que las tareas usen la CPU *a ráfagas*, dando la ilusión de que cada una tuviera su propia CPU. Si el quantum es muy corto, hay demasiados context switches y el sistema gasta mucho tiempo cambiando de tarea (overhead). Si es muy largo, las tareas interactivas se sienten lentas. El equilibrio es un parámetro de diseño del scheduler.

4.3. Operaciones **yield()** y **scheduler()**

El kernel define dos funciones para implementar este mecanismo:

- **yield():** el thread corriente *abandona* la CPU. Marca su estado como **RUNNABLE** y dispara un context switch al *scheduler*.
- **scheduler():** selecciona el próximo thread a darle la CPU.

Esquema simplificado:

```
1 struct task* current; // current task descriptor
2
3 void yield() {
4     current->state = RUNNABLE;
5     switch_context(&current->ctx, &scheduler_ctx);
6 }
```



```

7
8 void scheduler() {
9     while (1) {
10         current = NULL;
11         struct task* task = select_runnable_task(); // scheduling algorithm
12         if (task) {
13             context_switch(&scheduler->ctx, &(task->ctx));
14         }
15     }
16 }

```

4.4. El timer y el preemptive multitasking

Un thread que no invoca `yield()` monopolizaría la CPU. Para evitarlo, el kernel programa el *timer* (reloj) para que genere interrupciones periódicas. En cada tick, la ISR del timer puede decidir invocar `yield()` si el quantum se agotó.

Definición

Preemptive multitasking (o *tiempo compartido*): forma de planificación en la que el kernel puede recuperar la CPU de una tarea sin la cooperación de ésta, gracias a las interrupciones del timer.

4.5. Pasos de un context switch

Cuando el SO decide quitar la CPU a una tarea y asignársela a otra:

1. Salvar los registros de la CPU correspondientes en el contexto del descriptor de la tarea actual.
2. Cambiar al stack de la nueva tarea.
3. Restaurar los registros de la nueva tarea desde su contexto.

Implementación típica de `context_switch` (escrita en assembly):

```

1 context_switch(struct context *current, struct context *new) {
2     save_ctx(current); // copia pc y sp del HW al contexto current
3     restore_ctx(new); // copia pc y sp del contexto new al HW
4 }

```

4.6. Esquema mínimo de un kernel

Las Notas dan un esquema simplificado de cómo se ven las funciones de manejo de tareas en el kernel. El *trap handler* central despacha según el motivo:

```

1 trap(trap_number) {
2     push_cpu_registers();
3     switch (trap_number) {
4         case TIMER:
5             ticks++;
6             set_next_timer_interrupt();
7             if (current_task_used_quantum())
8                 yield(); // preempt CPU
9             break;
10        case SYSCALL:
11            syscall();
12            break;
13        case DISK_IRQ:
14            wakeup(&disk_wait_queue);

```

```

15         break;
16         // ...
17     }
18     restore_cpu_registers();
19     return_from_trap();
20 }

```

Y las primitivas `suspend/wakeup` para bloqueo y desbloqueo:

```

1 suspend(struct wait_queue *wq) {
2     current->state = WAITING;
3     enqueue(wq, current);
4     yield();
5 }
6
7 wakeup(struct wait_queue *wq) {
8     struct task *task = dequeue(wq);
9     if (task) task->state = RUNNABLE;
10 }

```

5. Procesos zombie y el primer proceso del sistema

5.1. El par `exit/wait`

- `exit(exitcode)`: finaliza el proceso corriente con código de salida `exitcode`.
- `wait(&exitcode)`: el proceso corriente se bloquea hasta que finalice un hijo. El código de salida se copia en el parámetro `exitcode`.

Esto ofrece un mecanismo básico de sincronización y comunicación entre procesos: el padre puede saber cómo terminó el hijo. Por convención el código de salida 0 significa terminación normal; cualquier valor distinto, anormal.

5.2. El primer proceso del sistema

En sistemas tipo UNIX, el primer proceso lanzado es `init` (PID 1). Construido manualmente por el kernel al final del boot, su imagen inicial es `initcode`, que invoca `exec("init")`. A partir de ahí, `init` dispara los procesos del sistema:

```

initcode pid=1 // hardcoded en el kernel
| exec("init")
init pid=1
| fork(); child: exec("tty")
tty pid=2
| exec("login")
login pid=3
| exec("sh")
shell pid=4

```

Esto hace que el sistema mantenga un *árbol de procesos* con la relación padre/hijo, donde `init` es la raíz. Cuando un padre muere antes que sus hijos, los hijos son adoptados por `init`, que se encargará del `wait()` correspondiente.

6. Concurrency

6.1. Definiciones y por qué importa

Definición

Concurrencia: mecanismos en un sistema que posibilitan la ejecución de múltiples hilos de ejecución por medio de ejecución simultánea (*paralelismo*) o tiempo compartido (*time sharing*) con cambios de contexto. Estos mecanismos pueden generar diferentes secuencias de operaciones de manera no determinística.

Aún con un único hilo de ejecución secuencial, el mecanismo de interrupciones o señales genera diferentes secuencias de ejecución (*intercalados* o *interleavings*). Por ejemplo:

```
1 int v = 1;
2
3 // disparado por una señal
4 signal_handler() {
5     v += 1; // (1)
6 }
7
8 // thread principal
9 v *= 2; // (2)
10 print(v); // (3)
```

Hay tres interleavings posibles según cuándo llegue la señal:

1. Sin señal: [(2), (3)] imprime 2.
2. Señal entre (2) y (3): [(2), (1), (3)] imprime 3.
3. Señal antes de (2): [(1), (2), (3)] imprime 4.

Tres salidas distintas para el mismo programa: ese es exactamente el problema de la concurrencia.

6.2. Paralelismo vs. tiempo compartido

- **Paralelismo:** dos o más hilos se ejecutan *realmente* a la vez en CPUs distintas. Es el caso de los sistemas multicore o multiprocesador.
- **Tiempo compartido:** una única CPU se va alternando entre varios hilos vía cambios de contexto. Solo uno corre en cada instante, pero al usuario le parece que todos progresan.

En ambos casos hay concurrencia: hilos concurrentes son aquellos cuya relación de orden no está garantizada por el programa.

6.3. Recursos compartidos, regiones críticas y race conditions

Dos o más tareas *cooperativas* usan algún recurso compartido: una variable, un archivo, un dispositivo. En procesos con múltiples threads, lo más común son las variables compartidas (*shared memory*). En procesos separados, hay que pasar mensajes o usar shared memory explícita.

Definición

La **región crítica** es la porción de código que accede a un recurso compartido.

Definición

Una **condición de carrera** (*race condition*) surge cuando el comportamiento del sistema depende de la secuencia (*timing* o *interleaving*) de la ocurrencia de eventos externos que pueden generar flujos de ejecución no deseados.

Importante

La concurrencia genera trazas de ejecución que no están bajo el control del programador, sino del scheduler (*no determinismo*). En un sistema correcto, este no determinismo no debería alterar los objetivos de cómputo del programa. La existencia de una condición de carrera *hace observable* ese no determinismo.

Para impedir trazas no deseadas hace falta usar **mecanismos de sincronización**.

7. Locks y exclusión mutua

7.1. Definición

Definición

Un **lock** (o *mutex*) se asocia a un recurso para sincronizar su acceso de manera exclusiva. Provee **exclusión mutua**: cuando un thread adquirió el lock, los demás que lo requieran deberán esperar hasta que el primero lo libere.

Un lock típicamente expone dos operaciones: `acquire(lock)` y `release(lock)`. El patrón estándar es:

```
1 lock l = create_lock("recurso r");
2 // ...
3 acquire(&l); // espera a que el recurso este libre
4 use_resource(r); // region critica
5 release(&l); // libera
```

Costos. El uso de locks tiene consecuencias adversas: las operaciones se *secuencializan* y producen *contención* en los hilos que compiten por el recurso. Hay que minimizar la duración de la región crítica para reducir la pérdida de paralelismo.

7.2. Spinlocks: el primer intento

Un *spinlock* hace que cada thread espere usando la CPU en un ciclo (*espera ocupada*). Una primera implementación:

```
1 void acquire(int *lk) {
2     while (*lk)
3         ; // (1) busy wait
4     *lk = 1; // (2)
5 }
6
7 void release(int *lk) {
8     *lk = 0;
9 }
```

¿Por qué falla? Supongamos que el thread 1 ejecuta `acquire()` y está en la línea (1) con `*lk == 0`. Antes de la línea (2) ocurre una interrupción de reloj y el scheduler pasa al thread 2. El thread 2 también invoca `acquire()`, ve `*lk == 0` y avanza a la línea (2), entrando a la región crítica. Otra interrupción y el control vuelve al thread 1, que también ejecuta la línea (2) y entra a la región crítica. **No se pudo garantizar exclusión mutua**: el problema es que el lock en sí mismo es un recurso compartido y las operaciones `acquire` y `release` deberían ser *atómicas* (sin interrupciones en el medio).

7.3. Solución 1: deshabilitar interrupciones

En un monoprocesador podemos deshabilitar interrupciones en `acquire()`:

```
1 void acquire(int *lk) {
2     cli(); // disable interrupts
3     while (*lk) ;
4     *lk = 1;
5 }
6
7 void release(int *lk) {
8     *lk = 0;
9     sti(); // enable interrupts
10 }
```

Funciona en monoprocesadores. *No funciona en multiprocesadores*: `cli()` solo deshabilita interrupciones en la CPU que la ejecuta, no en las otras.

7.4. Solución 2: instrucciones atómicas

Las CPUs modernas multicore con SMP proveen instrucciones atómicas indivisibles, implementadas mediante bloqueo del bus de memoria, bloqueo de la línea de caché o, en monoprocesadores, deshabilitando interrupciones. Las más comunes son:

- **Compare and swap (CAS)**: `cas(addr, old, new)`: si la variable en `addr` contiene `old`, la modifica con `new`.
- **Test and set (TAS)**: `old = tas(addr, new)`: asigna `new` y retorna el valor anterior.
- **Fetch and add (FAA)**: `faa(addr, value)`: suma `value` atómicamente.

GCC provee `__sync_lock_test_and_set(&lock, value)` con implementación dependiente de la arquitectura. En RISC-V emite:

```
li a5, 1
amoswap.w a5, a5, (lock_address)
```

7.5. Spinlocks en multiprocesadores

Una implementación correcta de spinlock para SMP combina deshabilitar interrupciones, instrucción atómica y *barreras de memoria*:

```
1 void acquire(int *lk) {
2     cli();
3     while (__sync_lock_test_and_set(lk, 1) != 0)
4         ;
5     __sync_synchronize(); // memory barrier (fence)
6 }
7
8 void release(int *lk) {
9     __sync_lock_release(lk); // *lk = 0;
10    __sync_synchronize();
11    sti();
12 }
```

Por qué la barrera. En multiprocesadores con *modelo de memoria relajado* (lo habitual en x86, ARM, RISC-V), una escritura hecha por una CPU puede no ser visible inmediatamente en otra (queda en su caché L1). La barrera fuerza a la CPU a propagar la escritura a memoria

principal e invalidar las cachés ajenas, asegurando que el cambio del lock sea visible en todos los cores.

7.6. Sleep locks

Un spinlock solo es útil para esperas *cortas*: si la región crítica tarda mucho, gastar CPU haciendo busy-wait es un desperdicio. Cuando la espera puede ser larga, conviene *suspender* al thread y reactivarlo cuando la condición cambie. Esto es lo que hace un **sleep lock**, similar a una variable de condición:

- `sleep(task* t, wait_queue* q)`: pasa *t* a estado SLEEPING y lo encola en *q*.
- `wakeup(wait_queue* q)`: desencola una tarea y la pasa a RUNNABLE.

La implementación interna requiere de spinlocks porque la cola es a su vez un recurso compartido y `sleep/wakeup` pueden ejecutarse concurrentemente.

7.7. Modularidad y problemas del uso de locks

Importante

Los locks **no son modulares**. Un thread puede adquirir un lock en una función y liberarlo en otra; los efectos colaterales del uso de locks deben ser documentados explícitamente.

El uso de locks es propenso a tres problemas clásicos:

Self-lock o double lock. Si un thread intenta adquirir un recurso que él mismo ya posee, queda bloqueado para siempre. Solución: *locks recursivos*.

Deadlock (interbloqueo). Dos hilos esperan mutuamente un recurso del otro:

```
1 f() { g() {  
2     acquire(&l1); acquire(&l2);  
3     // ... // ...  
4     acquire(&l2); acquire(&l1); // (potencial deadlock)  
5     release(&l2); release(&l1);  
6     release(&l1); release(&l2);  
7 } }
```

Si el thread A ejecutando `f()` adquiere `l1` y el thread B ejecutando `g()` adquiere `l2`, ambos quedan esperando indefinidamente. Las cuatro condiciones para que exista posibilidad de deadlock son (*condiciones de Coffman*):

1. **Exclusión mutua**: los recursos se adquieren de forma exclusiva.
2. **Espera y retención**: un proceso retiene un recurso mientras espera por otro.
3. **No quita** (*preemption*): los recursos se liberan solo voluntariamente.
4. **Espera circular**: existe una cadena $P_0, P_1, \dots, P_{n-1}, P_i$ tal que cada uno espera por un recurso adquirido por $P_{(i+1) \bmod n}$.

La estrategia más habitual para prevenir deadlock es *romper la condición 4* estableciendo un orden total para la adquisición de locks: si todos los módulos del sistema toman `l1` antes que `l2`, jamás se forma una cadena circular.

Livelock. Dos o más hilos quedan en ciclos sin progresar en alguna computación útil. No están bloqueados (siguen ejecutando), pero ninguno avanza. Es típico de protocolos mal diseñados de *retry*.

8. Solución de Peterson

Edsger Dijkstra y Gary Peterson estudiaron en los años 70 cómo resolver el problema de la región crítica usando *solo* lectura y escritura sobre memoria compartida (sin instrucciones atómicas del hardware). El algoritmo de Peterson (1981) lo logra para dos threads:

```
1 bool flag[2] = {false, false};
2 int turn;
3
4 lock(i) { unlock(i) {
5     flag[i] = true; flag[i] = 0;
6     int other = (i==0)? 1: 0; }
7     turn = other;
8     while (flag[other] && turn == other)
9         ;
10 }
```

La idea: `flag[i]` expresa la *intención* del thread i de ingresar a la sección crítica; `turn` indica qué thread tendría la prioridad para ingresar en caso de empate. Anunciando el interés *antes* de ceder el turno, el algoritmo garantiza exclusión mutua y progreso (la demostración semi-formal está en el Ejercicio 10 del Práctico 3).

El algoritmo se generaliza a N threads (*algoritmo filter*), pero su valor en la práctica moderna es limitado:

Importante

La solución de Peterson **no funciona en sistemas multiprocesadores** sin barreras de memoria explícitas, debido a las cachés y los modelos de memoria relajados. Por eso los sistemas reales usan instrucciones atómicas del hardware (TAS, CAS, FAA) y construyen mutexes/spinlocks sobre ellas.

9. Semáforos

9.1. Definición

Definición

Un **semáforo** (Dijkstra, 1963) representa una cantidad de recursos disponibles. Es una primitiva de sincronización más general que los locks. Su API tiene tres operaciones:

- `sem_init(s, value)`: inicializa el valor del semáforo s .
- `sem_wait(s)` (P): si el valor de s es cero, suspende al invocante; si no, decrementa.
- `sem_signal(s)` (V , también `sem_post`): incrementa el valor de s y despierta a algún thread bloqueado.

Un semáforo n -ario (o *counting semaphore*) permite que hasta n threads accedan al recurso. Un semáforo binario (con $n = 1$) es equivalente a un **mutex**: garantiza exclusión mutua.

Internamente un semáforo se implementa con un contador y un sleep lock: si el contador llega a cero, los `wait()`s subsiguientes ponen al thread en una wait queue; un `signal()` desencola a uno y lo pasa a RUNNABLE.

9.2. El productor-consumidor con semáforos

El problema clásico: uno o más productores generan valores y los ponen en un buffer acotado de capacidad N ; uno o más consumidores extraen valores del buffer y los procesan. Productores y consumidores ejecutan concurrentemente.

```

1  buffer buf[N];
2  semaphore mutex = 1, full = N, empty = 0;
3
4  void producer() {
5      while (true) {
6          value = gen_value();
7          sem_wait(&full);
8          add_to_buffer(value);
9          sem_signal(&empty);
10     }
11 }
12
13 void consumer() {
14     while (true) {
15         sem_wait(&empty);
16         value = get_from_buffer();
17         sem_signal(&full);
18         process(value);
19     }
20 }
21
22 void add_to_buffer(value) {
23     sem_wait(&mutex);
24     // agregar value al buffer
25     sem_signal(&mutex);
26 }
27
28 item_type get_from_buffer() {
29     sem_wait(&mutex);
30     // extraer item
31     sem_signal(&mutex);
32 }

```

Idea intuitiva

Tres semáforos cumplen tres roles distintos:

- **full/empty**: *condición* (cuántos slots libres u ocupados hay).
- **mutex**: *exclusión mutua* sobre la modificación del buffer y los índices.

Las regiones críticas son lo más cortas posible para no serializar todo el pipeline.

10. Variables de condición y monitores

10.1. Variables de condición

Definición

Una **variable de condición** es una primitiva de sincronización que maneja una cola de tareas esperando una condición (estado lógico). Comúnmente se usa con un lock asociado al recurso. Su interfaz es:

- **wait(&cond, &lock)**: requiere que el invocante haya adquirido el lock. *Atómicamente* libera el lock, agrega el thread a la cola y lo suspende.
- **signal(&cond)**: invocada por un thread que cambió la condición. Despierta uno o todos los threads esperando.

Productor-consumidor usando variables de condición:

```

1  condition cond; mutex lock; buffer buf;
2

```



```

3 producer() { consumer() {
4   acquire(&lock); acquire(&lock);
5   while (buffer_is_full()) while (buffer_is_empty())
6     wait(&cond, &lock); wait(&cond, &lock);
7   produce(&buf); consume(&buf);
8   signal(&cond); signal(&cond);
9   release(&lock); release(&lock);
10 } }

```

Las operaciones `wait()/notify()` de Java corresponden exactamente a este mecanismo.

10.2. Monitores

Definición

Un **monitor** (Hansen y Hoare, 1974) es una construcción sintáctica de un lenguaje de programación que encapsula datos y las operaciones sobre ellos al estilo orientado a objetos. Los hilos solo pueden manipular los datos por medio de las operaciones del monitor. Es *thread safe*: los invariantes están protegidos ante concurrencia.

Implementación común: variables de condición + al menos un mutex que asegura la exclusión mutua entre operaciones concurrentes.

```

1 monitor PC {
2   condition full, empty;
3   data buffer[N];
4   int count = 0;
5
6   void put(data item) {
7     if (count == N) wait(full);
8     insert(buffer, item);
9     if (++count == 1) signal(empty);
10  }
11
12  data get() {
13    if (count == 0) wait(empty);
14    data item = remove(buffer);
15    if (--count == N-1) signal(full);
16    return item;
17  }
18 };
19
20 void producer() {
21   while (true) { data item = gen_item(); PC.put(item); }
22 }
23 void consumer() {
24   while (true) { data item = PC.get(); process(item); }
25 }

```

Idea intuitiva

Los monitores son la abstracción *modular* de la sincronización: desde afuera, el productor y el consumidor solo ven `put()` y `get()`. Toda la complejidad (mutex, variables de condición, cuándo `wait/signal`) queda encapsulada y el código cliente no se puede equivocar olvidándose un `release`.

Java soporta monitores con la palabra clave **synchronized**: una clase con métodos sincronizados es un monitor implícito. Cada objeto Java tiene asociado un mutex y una variable de condición disponibles vía `wait()/notify()/notifyAll()`.

11. Mensajes y comunicación

11.1. Mensajes en lugar de memoria compartida

Los mecanismos vistos hasta acá se basan en *shared memory*. Cuando los procesos no comparten memoria (corren en computadoras distintas o el SO no les permite memoria compartida), se necesita *pasaje de mensajes*. El modelo se basa en dos primitivas:

- `send(destination, message)`: envía un mensaje a un proceso identificado.
- `receive(source, &message)`: recibe un mensaje del emisor (puede ser `any`).

La comunicación puede ser:

- **Asincrónica**: hay un buffer de mensajes (*mailbox*). El emisor no se bloquea; el receptor solo se bloquea si el mailbox está vacío.
- **Sincrónica** (*rendezvous*): no hay buffer. Emisor y receptor deben coincidir en `send/receive`.

11.2. Micro-kernels

En SOs con diseño *micro-kernel* como MINIX, los subsistemas (gestor de procesos, memoria, sistemas de archivos) se implementan como *servers* que reciben requerimientos por mensajes:

```
1 int main(void) {
2     message m;
3     init_subsystem();
4     while (TRUE) {
5         receive(ANY, &m);
6         switch (m.type) {
7             case REQUEST_1:
8                 // ...
9                 send(other_subsystem, m2);
10                break;
11                // ...
12            }
13        }
14    }
```

12. Futuros, promesas y concurrencia sin locks

12.1. Futuros y promesas

Definición

Un **futuro** (o *promesa*) es una abstracción que representa un valor que será computado por algún otro componente de ejecución concurrente. Se usan para implementar *data-flow variables*: variables que ante una lectura esperan a que se asigne su valor. La operación de lectura actúa como mecanismo de sincronización.

- `future f = async g(args)`: invocación asincrónica de `g()` en un nuevo thread. El invocante continúa ejecutando.
- `value = f.get()`: obtiene el valor encapsulado en `f`, posiblemente bloqueándose hasta que el thread que evalúa `g()` finalice.

12.2. Estructuras lock-free y wait-free

Definición

Una estructura de datos es **lock-free** si puede usarse concurrentemente sin operaciones de bloqueo. Es **wait-free** si cada thread que la usa puede completar sus operaciones en un número acotado de pasos.

Un spinlock califica como lock-free pero no como wait-free. Las estructuras lock-free generalmente requieren instrucciones atómicas y tienen ventajas de rendimiento (no hay contención por locks) y seguridad (si un thread finaliza con un lock adquirido, los demás no quedan bloqueados). Ejemplo de **push** lock-free a una lista enlazada usando CAS:

```
1 push(list* head, T value) {
2     list_node *n = create_node(value);
3     n->next = head->first;
4     while (!cas(&(head->first), n->next, n))
5         ;
6 }
```

13. IPC en UNIX

Los sistemas tipo UNIX ofrecen una variedad de mecanismos para la comunicación y sincronización entre procesos:

Mecanismo	Llamadas al sistema
File	open, read, write, close, ...
Pipes	pipe, read, write, close, ...
Named pipes	mkfifo, read, write, close, ...
Signals	kill, signal, alarm, ...
Message queues	msgget, msgrcv, msgctl, ...
Semaphores	semget, semctl, semop, sem_open, sem_wait, sem_post, ...
Shared memory	shmget, shmat, shmctl, shm_open, mmap, ...
Memory mapped files	mmap, munmap, ...
Sockets	listen, connect, accept, send, recv, ...

Todos estos mecanismos permiten la comunicación entre procesos locales, salvo los *sockets*, que también permiten la comunicación entre procesos en computadoras distintas (red).

14. Resumen final

1. Un **proceso** es una instancia de un programa en ejecución, con espacio de memoria propio. Un **thread** es la mínima unidad planificable: comparte memoria con otros threads del mismo proceso, pero tiene pila propia.
2. El SO mantiene por cada tarea un **descriptor** (PCB) con su PID, estado, mapa de memoria, recursos y *saved context*. Las tareas en **RUNNABLE** están en cola de listos; las **SLEEPING**, en wait queues asociadas al evento por el que esperan.
3. Las transiciones de estado se disparan por syscalls, interrupciones (timer, dispositivos) y excepciones. El timer es el mecanismo central del **preemptive multitasking**: permite al kernel recuperar la CPU sin la cooperación del proceso.
4. Un **cambio de contexto** guarda los registros del thread saliente y restaura los del entrante. La operación clave es `context_switch(current, new)` y suele escribirse en assembly.

5. La **conurrencia** es no determinismo controlado por el scheduler. Cuando varias tareas comparten un recurso, los *interleavings* pueden producir **condiciones de carrera**, que se evitan protegiendo las regiones críticas con primitivas de sincronización.
6. Hay una jerarquía de primitivas:
 - *Spinlocks*: para regiones críticas cortas. Usan instrucciones atómicas y barreras de memoria en multiprocesadores.
 - *Sleep locks*: para regiones largas. Suspenden al thread y lo despiertan con **wakeup**.
 - *Semáforos*: generalización del lock; el contador modela cantidad de recursos.
 - *Variables de condición*: hilos esperan que otra tarea cambie un estado lógico.
 - *Monitores*: encapsulación modular de los anteriores en una clase *thread safe*.
7. El uso de locks introduce problemas: *contención*, *deadlock* (cuatro condiciones de Coffman), *livelock*, *self-lock*. La estrategia más habitual para prevenir deadlock es definir un orden total de adquisición.
8. El algoritmo de **Peterson** demuestra que la exclusión mutua entre dos threads puede resolverse solo con lecturas y escrituras a memoria compartida, pero requiere atomicidad y consistencia secuencial; en la práctica los sistemas usan instrucciones atómicas del hardware.
9. El **productor-consumidor** con tres semáforos (**full**, **empty**, **mutex**) es el ejemplo prototípico de combinar sincronización por condición y exclusión mutua.
10. Cuando los procesos no comparten memoria, se comunican por **mensajes** (sincrónicos o asincrónicos). UNIX ofrece un catálogo amplio de mecanismos IPC: pipes, FIFOs, signals, message queues, semáforos, shared memory, sockets.

Importante

La idea unificadora: la concurrencia es no determinismo controlado, y el rol del SO es ofrecer abstracciones suficientes (estados, scheduler, locks, semáforos, monitores, mensajes) para que ese no determinismo no altere los objetivos de cómputo de los programas.